

TUTORIAL PASO A PASO · DESDE CERO · WINDOWS + NODE.JS

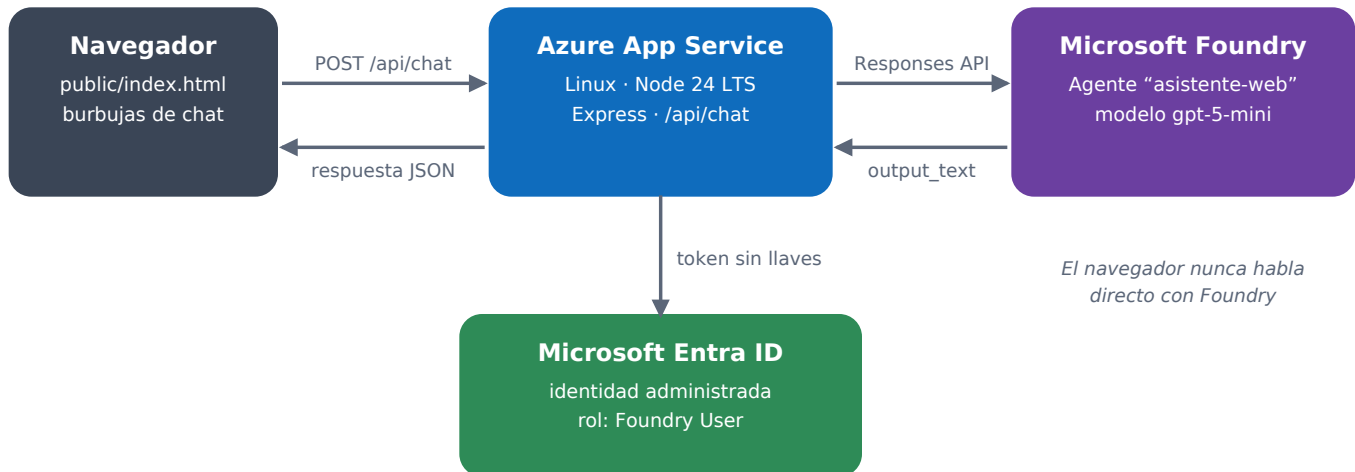
Tu página con chatbot construida por agentes

GitHub Copilot en modo Plan + copilot-instructions + harness de agentes,
un agente de Microsoft Foundry y despliegue en Azure App Service.

Resultado	https://<tu-app>.azurewebsites.net con un chatbot funcionando
Stack	Node.js 24 · Express 5 · HTML/JS · @azure/ai-projects 2.x · Azure CLI
Entorno	Windows 11 · PowerShell 7 · VS Code
Nivel	Intermedio · 90-120 minutos · Septiembre 2026

Qué vas a construir

Una página web con un chatbot que responde con un **agente de Microsoft Foundry**, publicada en **Azure App Service**. Lo interesante no es solo el resultado, sino el *cómo*: en lugar de pedirle a Copilot “hazme una página” y cruzar los dedos, vas a prepararle el terreno como a un equipo real: reglas del proyecto, roles especializados y un plan revisado antes de escribir una sola línea.



Piensa en la arquitectura como un restaurante: el navegador es el comensal, App Service es el mesero (recibe el pedido y lo lleva a la cocina) y Foundry es la cocina. El comensal nunca entra a la cocina, y el mesero no necesita llave: su gafete (la identidad administrada) le abre la puerta.

Las 4 fases

Fase	Qué haces	Pasos
A · Foundry	Proyecto, modelo desplegado y permisos	1
B · Preparar a Copilot	Repo, instrucciones y harness de agentes personalizados	2 – 6
C · Construir con modo Plan	/plan, revisión del plan, implementación y prueba local	7 – 10
D · Desplegar	App Service, identidad administrada, rol y verificación	11 – 12

Cómo usar este tutorial

Todo el código final viene completo en el **Paso 9**. Úsalo como punto de control: si lo que genera Copilot difiere, compara contra esa versión en lugar de empezar de nuevo.

Requisitos previos (Windows)

Necesitas	Para qué	Cómo verificar (PowerShell)
Windows 10/11 + PowerShell 7	Terminal para todo el tutorial	<code>\$PSVersionTable.PSVersion</code>
Node.js 24 LTS (incluye npm)	Backend Express (mismo runtime que App Service)	<code>node -v ; npm -v</code>
Azure CLI (az)	Login y despliegue	<code>az version</code>
VS Code actualizado + GitHub Copilot	Modo Plan, instrucciones y agentes	Chat view: <code>Ctrl+Alt+I</code>
Git + cuenta de GitHub	Versionar el proyecto	<code>git --version</code>
Suscripción de Azure	Foundry y App Service	<code>az account show</code>
Rol en Foundry	"Foundry User" (antes "Azure AI User") o superior sobre el recurso	Portal → IAM

Si te falta algo, instálalo con **winget** desde PowerShell y **cierra y vuelve a abrir la terminal** para que se actualice el PATH:

PowerShell (como usuario normal)

```
winget install --id Microsoft.PowerShell -e
winget install --id OpenJS.NodeJS.LTS -e
winget install --id Microsoft.AzureCLI -e
winget install --id Git.Git -e
winget install --id Microsoft.VisualStudioCode -e
```

Error típico de Windows

Si al correr npm ves "la ejecución de scripts está deshabilitada en este sistema", permite scripts firmados para tu usuario (una sola vez): `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Windows aquí, Linux allá

Tu PC es Windows, pero App Service correrá la app en **Linux**. No pasa nada: Node.js es multiplataforma y el código es el mismo. Es como escribir una carta en Word en tu PC y leerla en un Mac: el contenido no cambia, solo quién la abre.

Conceptos clave en 3 minutos

1 · Modo Plan: primero el plano, luego los ladrillos

En el modo Plan, Copilot investiga tu proyecto, te hace preguntas y te entrega un plan revisable **antes** de tocar código. Se activa escribiendo `/plan` seguido de lo que quieres, o eligiendo **Plan** en el selector de modo. Cuando termina aparece una tarjeta **Review Plan** con acciones como **Implement Plan**, **Approve Plan Only**, **Implement with Autopilot** o **Reject**.

Analogía: es la diferencia entre decirle a un albañil “hazme un cuarto” y enseñarle el plano firmado. El plano cuesta 5 minutos; rehacer una pared, una tarde.

2 · Instrucciones: el reglamento que Copilot lee siempre

Archivo	Alcance	Cuándo se aplica
<code>.github/copilot-instructions.md</code>	Todo el proyecto	Siempre, en cada petición
<code>.github/instructions/*.instructions.md</code>	Un tipo de archivo o tarea	Cuando el patrón <code>applyTo</code> coincide o la <code>description</code> es relevante
AGENTS.md (raíz)	Todo el proyecto, formato multi-agente	Lo leen Copilot, Codex y otros harnesses compatibles

Analogía: `copilot-instructions.md` es el reglamento interno de la empresa; los `.instructions.md` son los manuales de cada área (cocina, caja, limpieza) que solo lee quien trabaja ahí.

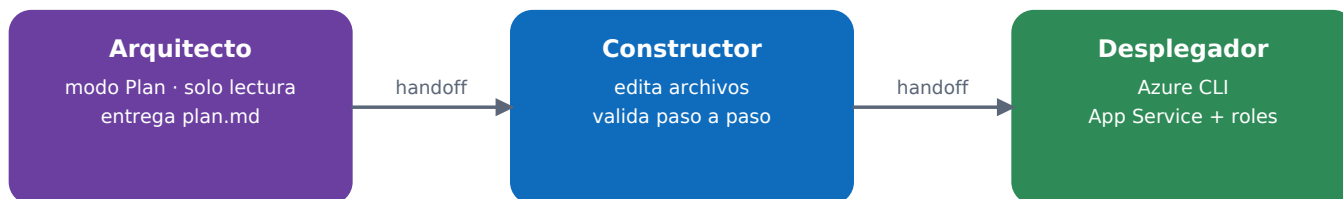
3 · Harness de agentes: el “cuerpo” que mueve al modelo

En VS Code, el **agent harness** es la capa de software que ejecuta la sesión: prepara el contexto y las instrucciones, le pasa las herramientas al modelo, aplica permisos y aprobaciones, y guarda el estado de la conversación. El modelo decide *qué* hacer; el harness se encarga de que *ocurra*. Lo eliges con el control **Session Target**:

Session Target	Qué es	Dónde trabaja
Local	Harness integrado de VS Code	Tu máquina (extension host)
Copilot / Claude / Codex	Harness del proveedor con sus capacidades propias	Tu máquina, un host remoto o un Dev Container
Cloud	Agente en la nube del proveedor	Repo de GitHub; entrega un Pull Request

Dentro de un harness defines **agentes personalizados** (`.github/agents/*.agent.md`): cada uno combina instrucciones, herramientas permitidas y opcionalmente un modelo. Con handoffs los encadenas: al terminar uno, aparece un botón para pasarle el trabajo al siguiente con el contexto incluido.

Analogía: el modelo es el cerebro; el harness es el cuerpo y el sistema nervioso; los agentes personalizados son los uniformes que se pone ese cuerpo según el turno (arquitecto, albañil, instalador).



Harness (Session Target: Copilot) · contexto común: copilot-instructions.md + *.instructions.md

PASO 1

Crear el proyecto de Foundry y desplegar el modelo

Fase A · Foundry

1. Entra a **ai.azure.com** (portal de Microsoft Foundry) con tu cuenta de Azure.
2. Crea un **proyecto nuevo**. El asistente crea también el recurso de Foundry que lo contiene. Anota el **nombre del recurso** y su **grupo de recursos**: los usarás en el Paso 11.
3. En el catálogo de modelos, despliega **gpt-5-mini** y deja el nombre del despliegue como **gpt-5-mini** (si usas otro nombre, cámbialo en tu `.env`).
4. En la página de inicio del proyecto copia el **endpoint del proyecto**. Tiene esta forma:
`https://<recurso>.services.ai.azure.com/api/projects/<proyecto>`
5. Verifica que tu usuario tenga el rol **Foundry User** (antes *Azure AI User*) sobre el recurso. Si tú creaste el recurso normalmente ya tienes un rol superior.

Atajo

Si solo quieres probar rápido, Foundry ofrece **modelos instantáneos** (preview) que no requieren despliegue, disponibles en proyectos de **West US 3**; ahí usas **gpt-5-mini** directo como nombre de modelo.

Ojo

Los nombres exactos de menús del portal cambian seguido. Si no ves un botón con el mismo nombre, busca la misma acción (crear proyecto, desplegar modelo, endpoint) en el menú lateral.

PASO 2

Crear el repositorio

Fase B · Preparar a Copilot

PowerShell

```

mkdir chatbot-foundry; cd chatbot-foundry
git init
New-Item -ItemType Directory -Force .github\instructions, .github\agents, public, scripts | Out-Null
az login
code .
  
```

Crea estos dos archivos a mano (son los únicos que no le pides a Copilot, porque definen qué no se sube a Git y dónde van tus valores locales):

Archivo: .aitianore

```
node_modules/
.env
.azure/
*.log
```

Archivo: .env.example

```
# Copia este archivo como .env y reemplaza los valores
FOUNDRY_PROJECT_ENDPOINT=https://<tu-recurso>.services.ai.azure.com/api/projects/<tu-proyecto>
FOUNDRY_AGENT_NAME=asistente-web
FOUNDRY_MODEL_DEPLOYMENT=gpt-5-mini
```

PowerShell

```
Copy-Item .env.example .env
notepad .env      # pega tu endpoint real del Paso 1 y guarda
```

Menos dependencias

No necesitas el paquete dotenv: Node 22.9+ lee el archivo con la bandera `--env-file-if-exists=.env`, que usaremos en el script `npm run dev`.

PASO 3**Escribir .github/copilot-instructions.md**

Fase B · Preparar a
Copilot

Este es el archivo más importante del tutorial. Todo lo que Copilot genere después va a respetar estas reglas: stack, versiones, arquitectura, seguridad y, muy importante en Windows, **que los comandos sean de PowerShell**. Sin esa línea, es común que el agente te proponga comandos de bash (`export`, \ al final de línea) que fallan en tu terminal.

Archivo: .github/copilot-instructions.md

```
# Instrucciones del proyecto: Chatbot Foundry en App Service

## Qué es este proyecto
Página web de una sola vista con un chatbot. El frontend (HTML/CSS/JS sin frameworks) llama a un backend Node.js con Express, y el backend conversa con un agente de Microsoft Foundry. Se despliega en Azure App Service (Linux, Node 24 LTS).

## Entorno de desarrollo
- Windows 11 con PowerShell 7. Todos los comandos de terminal que propongas deben ser de PowerShell (variables `$VAR`, continuación de línea con backtick).
- Node.js 24 LTS y npm. Nada de TypeScript ni bundlers.

## Stack y versiones
- Express 5, ES Modules (`"type": "module"`), `async/await`.
- SDK de Foundry: `@azure/ai-projects` 2.x (API "Foundry projects (new)"). No uses `@azure/ai-agents` ni la API 1.x.
- Autenticación: `DefaultAzureCredential` de `@azure/identity`.
- Variables de entorno en local con `node --env-file-if-exists=.env` (sin dotenv).

## Arquitectura (no la cambies sin preguntar)
- `server.js`: sirve `public/` en `/`, expone `POST /api/chat` y `GET /health`.
- El navegador NUNCA habla directo con Foundry. Solo el backend.
- Conversación multi-turno: `openai.conversations.create()` y `openai.responses.create({ conversation, input }, agentRef)`, donde `agentRef = { body: { agent: { name, type: "agent_reference" } } }`. El `conversation_id` viaja entre frontend y backend en el JSON.
- El servidor escucha en `process.env.PORT` (App Service lo define).
```

Seguridad

- Nunca escribas llaves, tokens ni endpoints reales en el código. Todo sale de variables de entorno: `FOUNDRY\_PROJECT\_ENDPOINT`, `FOUNDRY\_AGENT\_NAME`.
- En Azure se usa identidad administrada (managed identity), no API keys.
- Valida la entrada (1 a 2000 caracteres) y responde 422 si no cumple.
- Si Foundry falla, responde 502 con un mensaje genérico; no filtres trazas.

Estilo de código

- Comentarios en español neutro; nombres de variables en inglés.
- Cambios mínimos y enfocados: no reescribas archivos completos si basta editar.

Cómo validar

- Local: `npm run dev` y abrir <http://localhost:3000>
- Salud: `Invoke-RestMethod http://localhost:3000/health` debe devolver `status: ok`
- Antes de desplegar, confirma que `package.json` incluye todo lo que se importa y que el script `start` es `node server.js`.

Tip

Puedes pedirle a Copilot un primer borrador desde el editor de personalizaciones (**Chat: Open Customizations** → Overview) o con `/init` en una sesión Local, pero en un repo vacío no hay mucho que analizar: aquí conviene escribirlo tú.

PASO 4**Instrucciones por área con applyTo**

Fase B · Preparar a
Copilot

Divide las reglas específicas en archivos con frontmatter. VS Code los adjunta solo cuando el agente crea o modifica archivos que coinciden con `applyTo`, así no gastas contexto en reglas de CSS mientras editas el servidor.

Archivo: [.github/instructions/backend.instructions.md](#)

```
---
name: 'Backend Express'
description: 'Usar al crear o modificar el backend Node.js que llama a Microsoft Foundry.'
applyTo: 'server.js,scripts/**'
---
# Reglas del backend

* Crea el `AIPProjectClient` y el cliente de OpenAI una sola vez al iniciar, no en cada request.
* Referencia al agente en cada `responses.create` con
  `{ body: { agent: { name: AGENT_NAME, type: "agent_reference" } } }`.
* Usa `async/await` y envuelve las llamadas al SDK en `try/catch` → 502 genérico.
* Resuelve rutas de archivos con `import.meta.dirname`, no con el directorio actual.
* No agregues dependencias nuevas sin mencionarlas en el plan.
```

Archivo: [.github/instructions/frontend.instructions.md](#)

```
---
name: 'Frontend del chat'
description: 'Usar al crear o modificar la interfaz web del chatbot.'
applyTo: 'public/**'
---
# Reglas del frontend

* Solo HTML, CSS y JavaScript vanilla (ES2020+). Sin frameworks ni CDNs.
* Inserta el texto del bot con `textContent`, nunca con `innerHTML` (evita XSS).
* Guarda el `conversation_id` en una variable de JS; se reinicia al recargar.
* Deshabilita el botón de enviar mientras espera respuesta y muestra "Escribiendo...".
```

- * Diseño responsivo: debe verse bien a 360 px de ancho.
- * Usa variables CSS en `:root` para colores y soporta `prefers-color-scheme: dark`.

PASO 5**Armar el harness de agentes**

Fase B · Preparar a Copilot

5.1 · Elige el harness. En el Chat view abre **New Chat (+)** y en **Session Target** selecciona **Copilot**. Es el harness que ofrece el flujo dedicado de modo Plan con la tarjeta Review Plan y el archivo de plan editable.

5.2 · Crea tres agentes personalizados en `.github\agents\`. Puedes usar **Chat: New Custom Agent** o crearlos a mano. Cada uno tiene un rol, un nivel de permisos y un botón de handoff hacia el siguiente.

Agente	Herramientas	Responsabilidad	Handoff
Arquitecto	Solo lectura (búsqueda y web)	Plan en Markdown, sin editar	→ Constructor
Constructor	Todas las del harness	Implementar paso a paso y validar	→ Desplegador
Desplegador	Todas (usa la terminal)	Azure CLI en PowerShell con confirmación	—

Archivo: [.github/agents/arquitecto.agent.md](#)

```

---
name: Arquitecto
description: Investiga el repo y produce un plan de implementación. No edita archivos.
tools: ['search/codebase', 'search/usages', 'web/fetch']
handoffs:
  - label: Construir con el Constructor
    agent: Constructor
    prompt: Implementa el plan aprobado de arriba, paso por paso.
    send: false
---
# Rol: Arquitecto

Eres el arquitecto del proyecto. Tu trabajo es planear, no programar.

1. Lee `.github/copilot-instructions.md` y respeta la arquitectura definida.
2. Revisa qué archivos existen y cuáles faltan.
3. Entrega un plan en Markdown con: Resumen, Archivos a crear/modificar,
   Pasos numerados, Variables de entorno, Riesgos y Cómo verificar.
4. Si algo es ambiguo, pregunta antes de cerrar el plan.

Nunca edites archivos ni ejecute comandos.
```

Archivo: [.github/agents/constructor.agent.md](#)

```

---
name: Constructor
description: Implementa un plan aprobado con cambios mínimos y verificables.
handoffs:
  - label: Desplegar a Azure
    agent: Desplegador
    prompt: La app ya funciona en local. Prepárala y despliégala a Azure App Service.
    send: false
---
# Rol: Constructor
```

Implementas el plan que te entregue el Arquitecto.

- * Trabaja un paso a la vez y di qué archivo tocas y por qué.
- * Sigue ``.github/copilot-instructions.md`` y los ``.instructions.md`` que apliquen.
- * Después de cada paso, corre la validación indicada (por ejemplo ``Invoke-RestMethod http://localhost:3000/health`` en PowerShell).
- * Si un paso falla dos veces, detente y explica el error en lugar de improvisar.
- * No cambies la arquitectura ni agregues dependencias que el plan no mencione.

Archivo: `.aithub/agents/desplegador.agent.md`

```
---
name: Desplegador
description: Despliega a Azure App Service con Azure CLI (PowerShell) e identidad administrada.
---
# Rol: Desplegador (DevOps)
```

Despliegas a Azure App Service (Linux, Node 24 LTS) usando Azure CLI desde PowerShell en Windows.

Reglas:

- * Antes de cada comando ``az``, muestra el comando y explica qué hace.
- * Usa sintaxis de PowerShell: variables ``$VAR`` y continuación de línea con backtick.
- * Pide confirmación antes de crear, borrar o reemplazar recursos en Azure.
- * Nunca pongas secretos en archivos; usa App Settings e identidad administrada.
- * Secuencia esperada:
 1. ``az webapp up`` (plan B1, runtime NODE:24-lts). App Service corre ``npm start``.
 2. App Settings: ``FOUNDRY_PROJECT_ENDPOINT``, ``FOUNDRY_AGENT_NAME``.
 3. Identidad administrada del sistema + rol "Foundry User" (antes "Azure AI User") sobre el recurso de Foundry.
 4. Reinicia y verifica ``https://<app>.azurewebsites.net/health``; si algo falla, revisa logs con ``az webapp log tail``.

¿Por qué así?

Solo el Arquitecto restringe tools, con nombres de herramientas de solo lectura. En los otros dos se omite a propósito: sin tools el agente usa las herramientas disponibles en el harness, y te evitas escribir nombres que varían entre versiones. Principio de mínimo privilegio donde más importa: el que planea no puede tocar nada.

PASO 6

Verificar que Copilot carga todo

Fase B · Preparar a
Copilot

1. Abre la paleta con **Ctrl+Shift+P** y ejecuta **Chat: Open Customizations**.
2. Con el harness **Copilot** seleccionado, revisa que aparezcan tus instrucciones y los tres agentes.
3. Haz una pregunta de prueba, por ejemplo *"¿Cómo pruebo el endpoint /health desde mi terminal?"*, expande **References** y confirma que se usó `copilot-instructions.md`. Debe responderte con `Invoke-RestMethod (PowerShell)`, no con `curl` de bash.
4. Haz tu primer commit: `git add . ; git commit -m "Contexto para Copilot"`.

Tip

Si algo no aparece, ejecuta **Developer: Open Agent Debug Logs**: ahí se ven los errores de carga (frontmatter mal formado, ubicación incorrecta, etc.).

PASO 7

Pedir el plan con /plan

Fase C · Construir

Abre un chat nuevo con Session Target **Copilot**. Elige un modelo con buen razonamiento para planear y escribe este prompt. Incluye resultado esperado, restricciones y cómo verificar: los tres ingredientes que la documentación recomienda para un buen plan.

Prompt en el chat

```
/plan Construye desde cero una página web con un chatbot.
Contexto: lee .github/copilot-instructions.md; el repo solo tiene configuración.
Trabajo en Windows con PowerShell 7 y Node.js 24 LTS.
Resultado esperado:
- package.json (ES Modules) con scripts start, dev y create-agent.
- server.js (Express 5) que sirva public/ y exponga POST /api/chat y GET /health,
  escuchando en process.env.PORT.
- /api/chat conversa con el agente de Microsoft Foundry cuyo nombre viene de
  FOUNDRY_AGENT_NAME, usando @azure/ai-projects 2.x
  (getOpenAIClient + conversations + responses con agent_reference).
- Frontend en public/ (index.html, styles.css, app.js) con burbujas de chat.
- scripts/create-agent.js que cree el agente con kind "prompt".
Restricciones: sin frameworks de frontend, sin TypeScript, sin dotenv, sin API keys.
Verificación: /health responde ok; una pregunta y un seguimiento mantienen el contexto;
un mensaje vacío devuelve 422.
Identifica las decisiones que debo tomar antes de implementar.
```

Copilot investigará el repo y probablemente te pregunte cosas como si el historial se guarda en el navegador o en el servidor, o qué hacer si Foundry falla. Contesta con base en tus instrucciones (por ejemplo: “*conversation_id* en una variable JS; si falla, 502 con mensaje genérico”).

Alternativa

¿Prefieres tu propio planificador? Selecciona el agente **Arquitecto** en el dropdown de agentes y pega el mismo prompt sin /plan. Obtienes un plan con tu formato y el botón **Construir con el Constructor** al final.

PASO 8

Revisar, corregir y aprobar el plan

Fase C · Construir

1. En la tarjeta **Review Plan** selecciona **Open Full Plan**. El plan se abre como Markdown editable: puedes cambiar texto o dejar comentarios en línea.
2. Revísalo con esta lista: **alcance** (solo lo pedido), **encaje** (archivos y patrones correctos), **decisiones** explícitas y **verificación** concreta.
3. Si algo no te convence, escribe feedback general en la tarjeta y pulsa **Submit Feedback**. Ejemplo:

```
No agregues autenticación de usuarios ni base de datos en esta versión.
Separa la implementación en pasos que se puedan probar de forma independiente.
Todos los comandos de verificación deben ser de PowerShell.
Agrega una verificación de que el frontend no usa innerHTML para las respuestas del bot.
```

4. Cuando el plan esté bien, elige cómo implementarlo:

Acción	Úsala cuando...
Implement Plan	Quieres implementarlo en la misma sesión, aprobando herramientas una a una.

Acción	Úsala cuando...
Approve Plan Only	Quieres cambiar a un modelo más económico para implementar, o pasarlo al agente Constructor : cambia de Plan a Interactive , elige el modelo y pide "implementa el plan aprobado".
Implement with Autopilot	Confías en el plan y en los permisos configurados; el agente sigue solo. Recomendado solo después de tu primera pasada manual.
Reject	El plan no tiene arreglo; empieza con un prompt mejor.

Ojo

Aprobar el plan y aprobar las herramientas son decisiones separadas. Revisa el nivel de permisos de la sesión antes de implementar: tú decides si cada comando de terminal pide confirmación.

PASO 9**Código de referencia (punto de control)**

Fase C · Construir

Esto es lo que debería quedar al terminar la implementación. Si Copilot generó algo distinto pero funciona y respeta las instrucciones, está bien; si algo falla, compara contra esto. Este servidor se probó con el SDK real @azure/ai-projects 2.7.1, simulando las respuestas HTTP de Foundry: /health, archivos estáticos, conversación multi-turno (con el JSON exacto que llega a Foundry), validación 422 y error 502.

Archivo: package.json

```
{
  "name": "chatbot-foundry",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "engines": {
    "node": ">=22.9"
  },
  "scripts": {
    "start": "node server.js",
    "dev": "node --env-file-if-exists=.env --watch server.js",
    "create-agent": "node --env-file=.env scripts/create-agent.js"
  },
  "dependencies": {
    "@azure/ai-projects": "^2.7.1",
    "@azure/identity": "^4.13.3",
    "express": "^5.2.1"
  }
}
```

Archivo: server.js

```
// Backend del chatbot: Express + agente de Microsoft Foundry
import path from "node:path";
import express from "express";
import { AIProjectClient } from "@azure/ai-projects";
import { DefaultAzureCredential } from "@azure/identity";

const ENDPOINT = process.env.FOUNDRY_PROJECT_ENDPOINT;
const AGENT_NAME = process.env.FOUNDRY_AGENT_NAME ?? "asistente-web";
const PORT = process.env.PORT ?? 3000; // App Service define PORT
```

```

if (!ENDPOINT) {
  console.error("Falta FOUNDRY_PROJECT_ENDPOINT (.env en local, App Settings en Azure)");
  process.exit(1);
}

// Un solo cliente para toda la app (local: az login / Azure: managed identity)
const project = new AIPProjectClient(ENDPOINT, new DefaultAzureCredential());
const openai = project.getOpenAIClient();
const agentRef = { body: { agent: { name: AGENT_NAME, type: "agent_reference" } } };

const app = express();
app.use(express.json({ limit: "16kb" }));
app.use(express.static(path.join(import.meta.dirname, "public"))); // "/" -> index.html

app.get("/health", (_req, res) => res.json({ status: "ok" }));

app.post("/api/chat", async (req, res) => {
  const { message, conversation_id: conversationIdIn } = req.body ?? {};
  if (typeof message !== "string" || !message.trim() || message.length > 2000) {
    return res.status(422).json({ detail: "El mensaje debe tener entre 1 y 2000 caracteres." });
  }

  try {
    // Primera pregunta: crea la conversación; las siguientes la reutilizan
    const conversationId = conversationIdIn || (await openai.conversations.create()).id;
    const response = await openai.responses.create(
      { conversation: conversationId, input: message },
      agentRef,
    );
    res.json({ reply: response.output_text, conversation_id: conversationId });
  } catch (err) {
    console.error("Error al llamar a Foundry:", err.message);
    res.status(502).json({ detail: "El asistente no está disponible." });
  }
});

app.listen(PORT, () => console.log(`Chatbot escuchando en http://localhost:${PORT}`));

```

¿Cómo sabe Foundry qué agente usar?

La línea clave es `agentRef`: el SDK fusiona `{ conversation, input }` con `{ agent: { name, type: "agent_reference" } }` en un solo JSON. Así Foundry sabe qué agente (instrucciones + modelo) debe responder.

Archivo: `scripts/create-agent.is`

```

// Crea (o actualiza) el agente de Microsoft Foundry que usará el chatbot.
// Ejecuta una sola vez: npm run create-agent (requiere az login y el archivo .env)
import { AIPProjectClient } from "@azure/ai-projects";
import { DefaultAzureCredential } from "@azure/identity";

const ENDPOINT = process.env.FOUNDRY_PROJECT_ENDPOINT;
const AGENT_NAME = process.env.FOUNDRY_AGENT_NAME ?? "asistente-web";
const MODEL = process.env.FOUNDRY_MODEL_DEPLOYMENT ?? "gpt-5-mini";

const INSTRUCTIONS = `Eres el asistente virtual de este sitio web.
Respondes en español, de forma breve (máximo 4 párrafos) y amable.
Si no sabes algo, dilo con honestidad y sugiere dónde buscar.
No inventes precios, fechas ni datos de contacto.`;

const project = new AIPProjectClient(ENDPOINT, new DefaultAzureCredential());

const agent = await project.agents.createVersion(AGENT_NAME, {

```

```

kind: "prompt",
model: MODEL,
instructions: INSTRUCTIONS,
});
console.log(`Agente listo -> nombre: ${agent.name}, versión: ${agent.version}`);

```

Archivo: `public/index.html`

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Asistente IA</title>
  <link rel="stylesheet" href="/styles.css">
</head>
<body>
  <header class="hero">
    <h1>Asistente IA</h1>
    <p>Hecho con GitHub Copilot, Microsoft Foundry y Azure App Service.</p>
  </header>

  <main class="chat" aria-live="polite">
    <div id="messages" class="messages">
      <div class="msg bot">¡Hola! ¿En qué te ayudo hoy?</div>
    </div>
    <form id="chat-form" class="composer">
      <input id="input" type="text" maxlength="2000"
        placeholder="Escribe tu pregunta..." autocomplete="off" required>
      <button id="send" type="submit">Enviar</button>
    </form>
  </main>

  <script src="/app.js"></script>
</body>
</html>

```

Archivo: `public/app.js`

```

const form = document.getElementById("chat-form");
const input = document.getElementById("input");
const sendBtn = document.getElementById("send");
const messages = document.getElementById("messages");

let conversationId = null; // se reinicia al recargar la página

function addMessage(text, who) {
  const div = document.createElement("div");
  div.className = `msg ${who}`;
  div.textContent = text; // textContent evita inyectar HTML (XSS)
  messages.appendChild(div);
  messages.scrollTop = messages.scrollHeight;
  return div;
}

form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const text = input.value.trim();
  if (!text) return;

  addMessage(text, "user");
  input.value = "";
  sendBtn.disabled = true;
  const typing = addMessage("Escribiendo...", "bot typing");

```

```

try {
  const res = await fetch("/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message: text, conversation_id: conversationId }),
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.detail || "Error del servidor");
  conversationId = data.conversation_id;
  typing.remove();
  addMessage(data.reply, "bot");
} catch (err) {
  typing.remove();
  addMessage(`Ups: ${err.message}`, "bot error");
} finally {
  sendBtn.disabled = false;
  input.focus();
}
});

```

Archivo: `public/styles.css`

```

:root {
  --bg: #f5f7fb; --card: #ffffff; --text: #1b2330; --muted: #5b6678;
  --accent: #0f6cbd; --user: #0f6cbd; --user-text: #ffffff; --bot: #eef2f8;
}
@media (prefers-color-scheme: dark) {
  :root { --bg: #11151c; --card: #1a2029; --text: #e6ebf2; --muted: #9aa6b8;
    --accent: #4ealff; --user: #2b6fd6; --bot: #252d39; }
}
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: var(--bg); color: var(--text); }
.hero { text-align: center; padding: 32px 16px 8px; }
.hero h1 { margin: 0 0 6px; }
.hero p { margin: 0; color: var(--muted); }
.chat { max-width: 720px; margin: 16px auto; padding: 0 16px; }
.messages { background: var(--card); border-radius: 12px; padding: 16px; height: 60vh;
  overflow-y: auto; display: flex; flex-direction: column; gap: 10px; }
.msg { max-width: 80%; padding: 10px 14px; border-radius: 14px; line-height: 1.45;
  white-space: pre-wrap; }
.msg.user { align-self: flex-end; background: var(--user); color: var(--user-text); }
.msg.bot { align-self: flex-start; background: var(--bot); }
.msg.typing { opacity: .7; font-style: italic; }
.msg.error { border: 1px solid #d13438; }
.composer { display: flex; gap: 8px; margin-top: 12px; }
.composer input { flex: 1; padding: 12px; border-radius: 10px; border: 1px solid var(--muted);
  background: var(--card); color: var(--text); font-size: 1rem; }
.composer button { padding: 12px 18px; border: 0; border-radius: 10px; background: var(--accent);
  color: #fff; font-weight: 600; cursor: pointer; }
.composer button:disabled { opacity: .5; cursor: wait; }

```

Estructura final del repositorio:

```

chatbot-foundry\
├── .github\
│   ├── copilot-instructions.md
│   └── instructions\
│       ├── backend.instructions.md
│       └── frontend.instructions.md
├── agents\
│   ├── arquitecto.agent.md
│   ├── constructor.agent.md
│   └── desplegador.agent.md
├── public\ (index.html, app.js, styles.css)
├── scripts\create-agent.js
└── server.js

```

```
├─ package.json
├─ .env.example  (.env y node_modules quedan fuera de git)
└─ .gitignore
```

PASO 10**Crear el agente en Foundry y probar en local**

Fase C · Construir

PowerShell (en la carpeta del proyecto)

```
npm install
npm run create-agent
# -> Agente listo -> nombre: asistente-web, versión: 1

npm run dev
# -> Chatbot escuchando en http://localhost:3000
```

Abre <http://localhost:3000> y prueba la memoria de la conversación: pregunta “¿Cuál es la capital de Japón?” y luego “¿Y cuántos habitantes tiene?”. Si la segunda respuesta entiende que hablas de Tokio, el `conversation_id` está viajando bien. Gracias a `--watch`, el servidor se reinicia solo cuando guardas cambios.

Pruebas rápidas desde otra terminal de PowerShell

```
Invoke-RestMethod http://localhost:3000/health

Invoke-RestMethod -Method Post -Uri http://localhost:3000/api/chat `
  -ContentType "application/json; charset=utf-8" `
  -Body (@{ message = "Hola, ¿quién eres?" } | ConvertTo-Json)
```

La magia sin llaves

`DefaultAzureCredential` usa tu sesión de `az login` en local y la identidad administrada en Azure. Mismo código, cero llaves: por eso no hay ninguna API key en el proyecto.

Haz commit y, si quieres, publica el repo en GitHub: `git add . ; git commit -m "Chatbot funcionando en local"`.

PASO 11**Desplegar a Azure App Service con el Desplegador**

Fase D · Desplegar

Pulsa el handoff **Desplegar a Azure** (o selecciona el agente **Desplegador**) y usa este prompt. El agente propondrá los comandos de abajo uno por uno y te pedirá confirmación; léelos antes de aprobar. Si prefieres hacerlo tú, cópialos directo en PowerShell.

Prompt para el Desplegador

```
Despliega esta app a Azure App Service (Linux, Node 24 LTS) en el grupo rg-chatbot-foundry,
región eastus2. El recurso de Foundry se llama <FOUNDRY_NAME> y está en el grupo <FOUNDRY_RG>.
Usa identidad administrada y el rol Foundry User. Muéstrame cada comando de PowerShell
antes de ejecutarlo.
```

PowerShell · despliegue completo

```
# 0) Variables (ajusta los valores entre < >)
$RG          = "rg-chatbot-foundry"
$LOCATION      = "eastus2"
$APP         = "chatbot-foundry-$(Get-Random -Maximum 99999)" # único en azurewebsites.net
$FOUNDRY_RG  = "<grupo-del-recurso-foundry>"
$FOUNDRY_NAME = "<nombre-del-recurso-foundry>"
$ENDPOINT    = "https://<recurso>.services.ai.azure.com/api/projects/<proyecto>"

# 1) Grupo de recursos + App Service (Linux, Node 24 LTS, plan B1) y primer despliegue
az group create --name $RG --location $LOCATION
az webapp up --name $APP --resource-group $RG --location $LOCATION `
  --runtime "NODE:24-lts" --sku B1

# 2) Variables de entorno de la app (App Settings)
az webapp config appsettings set -g $RG -n $APP --settings `
  FOUNDRY_PROJECT_ENDPOINT=$ENDPOINT FOUNDRY_AGENT_NAME=asistente-web

# 3) Identidad administrada del sistema
$PRINCIPAL_ID = az webapp identity assign -g $RG -n $APP --query principalId -o tsv

# 4) Rol Foundry User (antes Azure AI User) sobre el recurso de Foundry
$FOUNDRY_ID = az cognitiveservices account show -g $FOUNDRY_RG -n $FOUNDRY_NAME `
  --query id -o tsv
az role assignment create --assignee-object-id $PRINCIPAL_ID `
  --assignee-principal-type ServicePrincipal `
  --role "53ca6127-db72-4b80-b1b0-d745d6d5456d" --scope $FOUNDRY_ID

# 5) Reinicia para tomar configuración y permisos
az webapp restart -g $RG -n $APP
"https://$APP.azurewebsites.net"
```

Qué pasa en cada bloque

Bloque 1: `az webapp up` crea una app **Linux** por defecto, comprime la carpeta, la sube y ejecuta `npm install` en Azure. Luego arranca con `npm start` (por eso el script `start` es `node server.js`, sin `--env-file`). Es normal que falle justo después, porque aún no existe `FOUNDRY_PROJECT_ENDPOINT`; se arregla en los bloques 2 y 5. Si el runtime no se reconoce, consulta los disponibles con `az webapp list-runtimes --os linux -o table`.

Backtick (`): en PowerShell es el carácter de continuación de línea; debe ser lo último de la línea, sin espacios después.

Bloque 4: el GUID identifica el rol aunque cambie de nombre en el portal. Si tu CLI lo rechaza, asigna el rol desde el portal: recurso de Foundry → Control de acceso (IAM) → Agregar asignación de rol → *Foundry User* → Identidad administrada → tu App Service.

Ojo

La asignación de roles puede tardar algunos minutos en propagarse. Si ves 502 justo después del bloque 4, espera 2 a 5 minutos y vuelve a reiniciar.

Redespliegues

Para redeployar después de cambios basta con `az webapp up` desde la carpeta del proyecto: reutiliza los valores guardados en `.azure/config`. Si la subida es lenta, borra `node_modules` antes (`Remove-Item -Recurse -Force node_modules`); Azure lo reinstala.

PASO 12**Verificar en producción**

Fase D · Desplegar

```
Invoke-RestMethod "https://$APP.azurewebsites.net/health" # status: ok
az webapp log tail -g $RG -n $APP # logs en vivo si algo falla
```

Abre <https://<tu-app>.azurewebsites.net>, repite la prueba de las dos preguntas y listo: tu página con chatbot está en la nube, construida con un flujo de agentes que planearon, implementaron y desplegaron con tu supervisión.

Solución de problemas

Síntoma	Causa probable	Solución
"npm.ps1 no se puede cargar... ejecución de scripts deshabilitada"	Política de ejecución de PowerShell	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
"node" o "az" no se reconoce	La terminal no ve el PATH nuevo tras winget	Cierra y abre PowerShell (o VS Code completo)
Falta FOUNDRY_PROJECT_ENDPOINT	Sin .env en local o sin App Settings en Azure	Usa npm run dev en local (no npm start) o repite el bloque 2 del Paso 11
502 "El asistente no está disponible"	Sin permisos, agente inexistente o nombre distinto	Revisa el rol Foundry User, corre npm run create-agent y compara FOUNDRY_AGENT_NAME
401/403 en local	Sesión de az expirada o en otro tenant	az login (y az account set --subscription ...)
"Application Error" en Azure	No escucha en process.env.PORT o falta el script start	Revisa server.js y package.json; mira az webapp log tail
Copilot propone comandos de bash	No cargó las instrucciones o falta la sección Entorno	Chat: Open Customizations; revisa References y el harness
El agente no aparece en el dropdown	Archivo fuera de .github\agents o YAML inválido	Developer: Open Agent Debug Logs
No recuerda el contexto	conversation_id no se reenvía	Revisa app.js: debe guardar data.conversation_id y enviarlo en cada POST

Limpieza para no generar costos

El plan B1 de App Service y el uso del modelo tienen costo. Cuando termines de experimentar:

```
az group delete --name rg-chatbot-foundry --yes --no-wait
```

Si creaste el proyecto de Foundry solo para este tutorial, elimina también su grupo de recursos.

Siguientes pasos

- **CI/CD:** pídele al Arquitecto un plan para un workflow de GitHub Actions que despliegue en cada push a main usando OIDC (sin secretos).
- **Autenticación:** activa la autenticación integrada de App Service (Easy Auth) para que solo usuarios de tu organización usen el chat.
- **Herramientas del agente:** agrega búsqueda web o archivos propios al agente en Foundry para que responda sobre tu contenido.
- **Streaming:** usa `stream: true` en `responses.create` y Server-Sent Events para mostrar la respuesta mientras se genera.
- **AGENTS.md:** si tu equipo usa varios harnesses (Copilot, Codex, Claude), mueve las reglas comunes a un `AGENTS.md` compartido.

Fuentes oficiales

- VS Code · Plan work with agents: code.visualstudio.com/docs/agents/run/planning
- VS Code · Understand agent harnesses: code.visualstudio.com/docs/agents/concepts/agent-harnesses
- VS Code · Custom instructions: code.visualstudio.com/docs/agent-customization/custom-instructions
- VS Code · Custom agents: code.visualstudio.com/docs/agent-customization/custom-agents
- Microsoft Learn · Get started with Microsoft Foundry SDK: learn.microsoft.com/azure/foundry/quickstarts/get-started-code
- npm · @azure/ai-projects (README con ejemplos de agentes en JavaScript): npmjs.com/package/@azure/ai-projects
- Microsoft Learn · Create a Node.js web app in App Service: learn.microsoft.com/azure/app-service/quickstart-nodejs

Documentación consultada en septiembre de 2026. Los nombres de menús y versiones de SDK cambian con frecuencia; ante la duda, la documentación oficial manda.